NOVA SCHOOL OF SCIENCE & TECHNOLOGY

STR

LAB WORK 2: CLASS 3

Java Real-Time – Synchronization and Communication Mechanisms

2025 - 2026

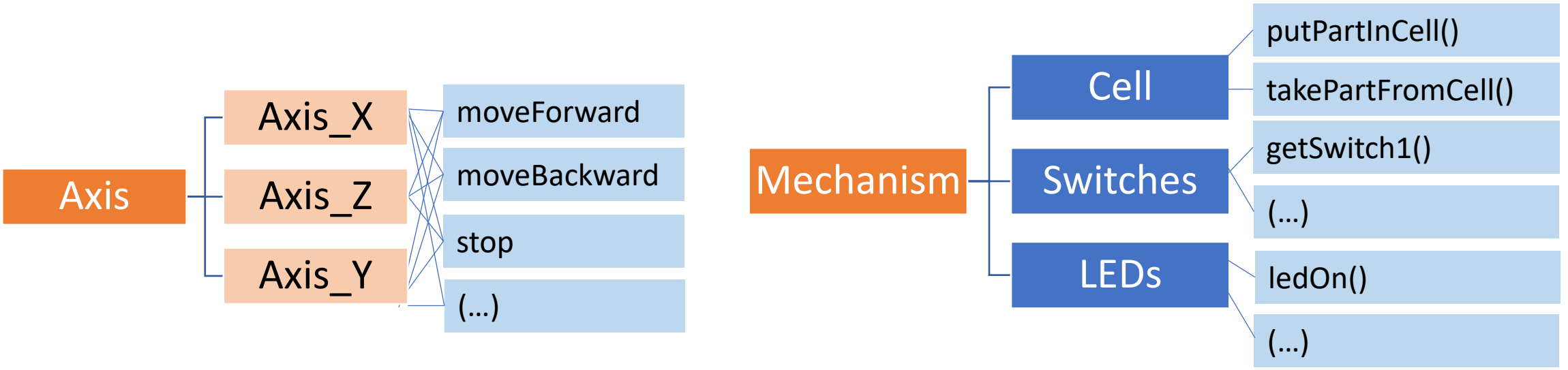
OBJECTIVES

- Some organization structure for Labwork2
- Why Threads?
- Threads – some aspects to consider
- Identification of needed Java Threads for Lab2
- Short-lived Threads
- Long-lived Threads
- Synchronization with a Semaphore
- Blocking queue example
- **CalibrationThread**

ORGANIZATION STRUCTURE

REVIEW

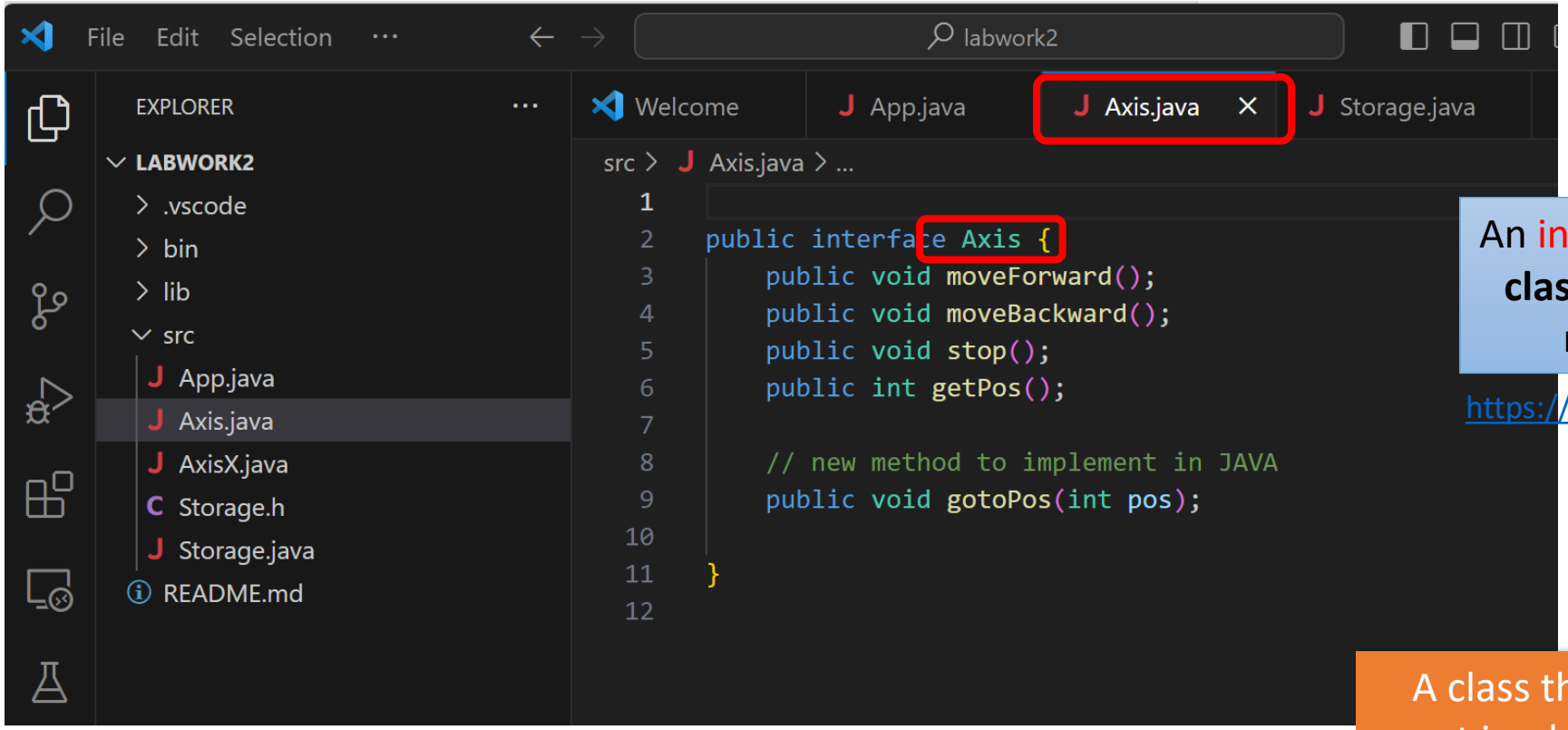
- Having the class **Storage.java** already developed (DLL), at the Java side we need to organize our classes into a more logical and straightforward way.
- This should be done by aggregating the storage functionality as several well identified classes, some of them as Java threads.
- For instance, we can define concrete classes for axis operation and a class named mechanism, which aggregates the functionality for the cell movements, the switches, and the LEDs.



THE AXIS INTERFACE

REVIEW

Let's start by providing an interface definition for the Axis: Java interface -> Axis.java



An interface is a completely **"abstract class"** that is used to group related methods with empty bodies

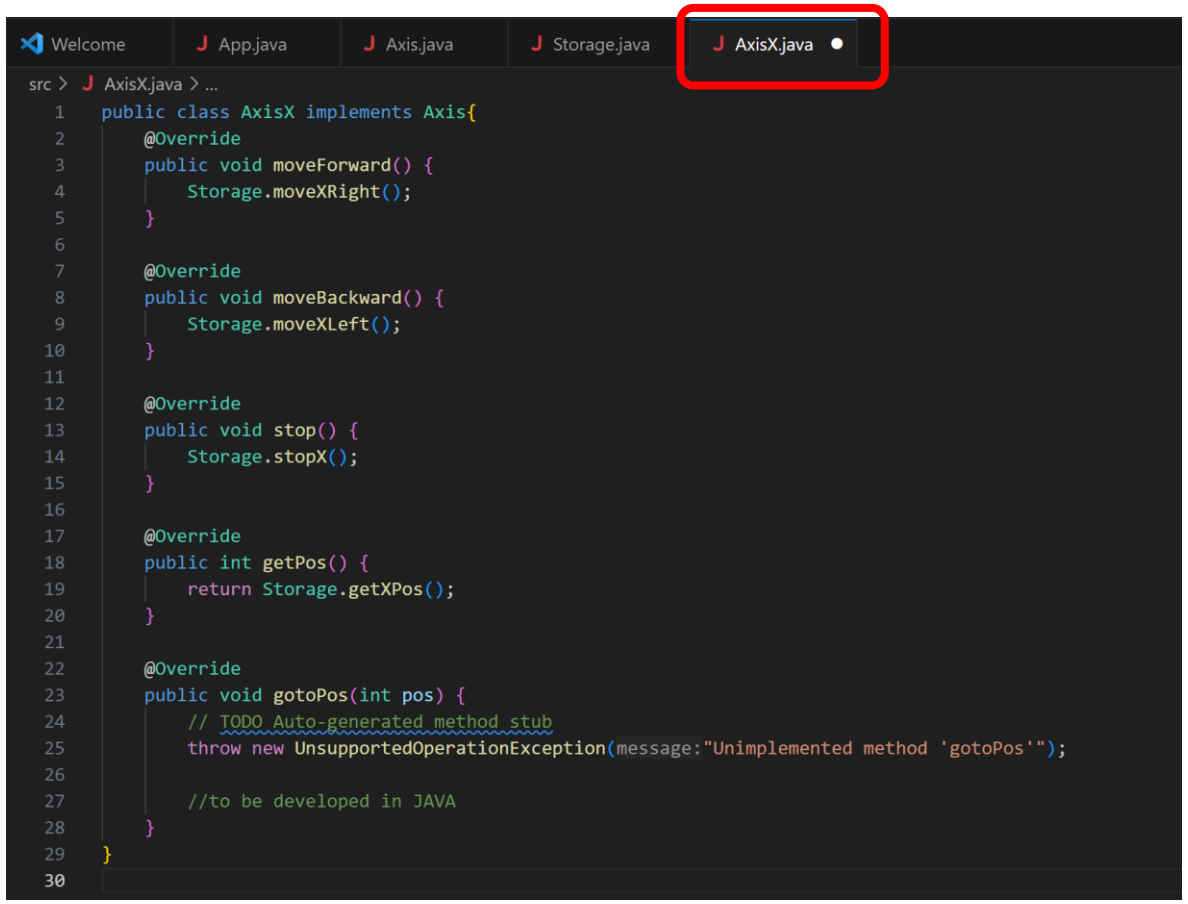
https://www.w3schools.com/java/java_interface.asp

A class that implements a specific interface must implement the methods of the interface.

The interface has several methods that are used in each axis.
Although each axis has distinct implementation, in an abstract way they behave similarly…..
As such, interface **Axis** specifies and describes the behavior of each axis in an abstract way.

CLASS AXISX, AXISZ and AXISY

REVIEW



The same for AxisZ.java and AxisY.java

Each concrete **axis** can follow the **Axis interface** and implement the concrete functionality of each Axis's methods.

When a class implements an axis, it must **implement all** the interface's methods.

Based on the given interface, try develop **three classes** for each **axis** existing in the physical system.

THE CLASS MECHANISM

REVIEW

Create a new class Mechanism.java

The class mechanism deals with:
- the LEDs
- the switches
- the movements to insert and take parts from cell

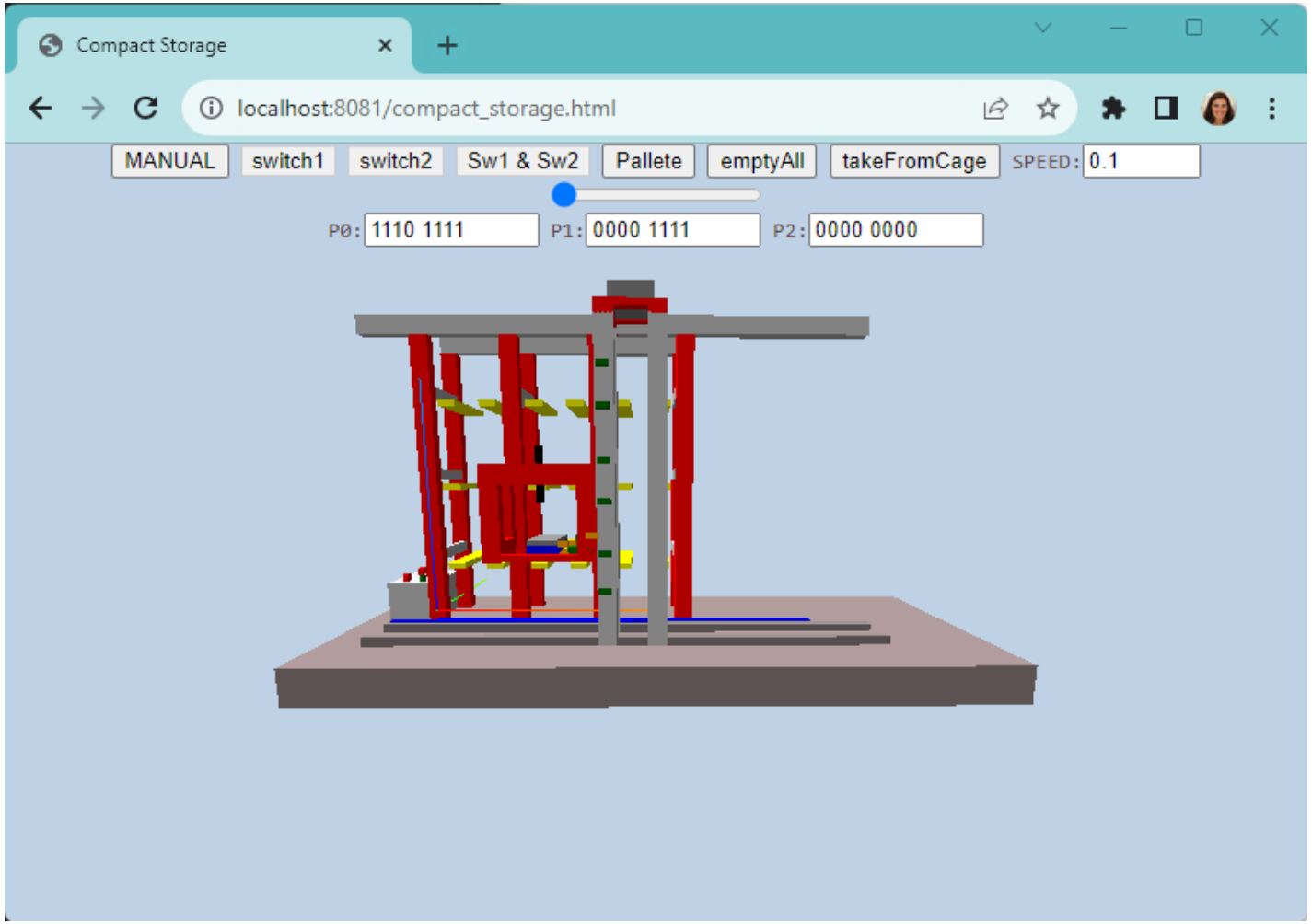
Remember, some of these methods might be static, others not

Static methods do not use any instance variables of any object of the class they are defined in. Static methods take all the data from parameters and compute something from those parameters, with no reference to variables.

https://www.tutorialspoint.com/Java-static-method

WHY THREADS?

REVIEW



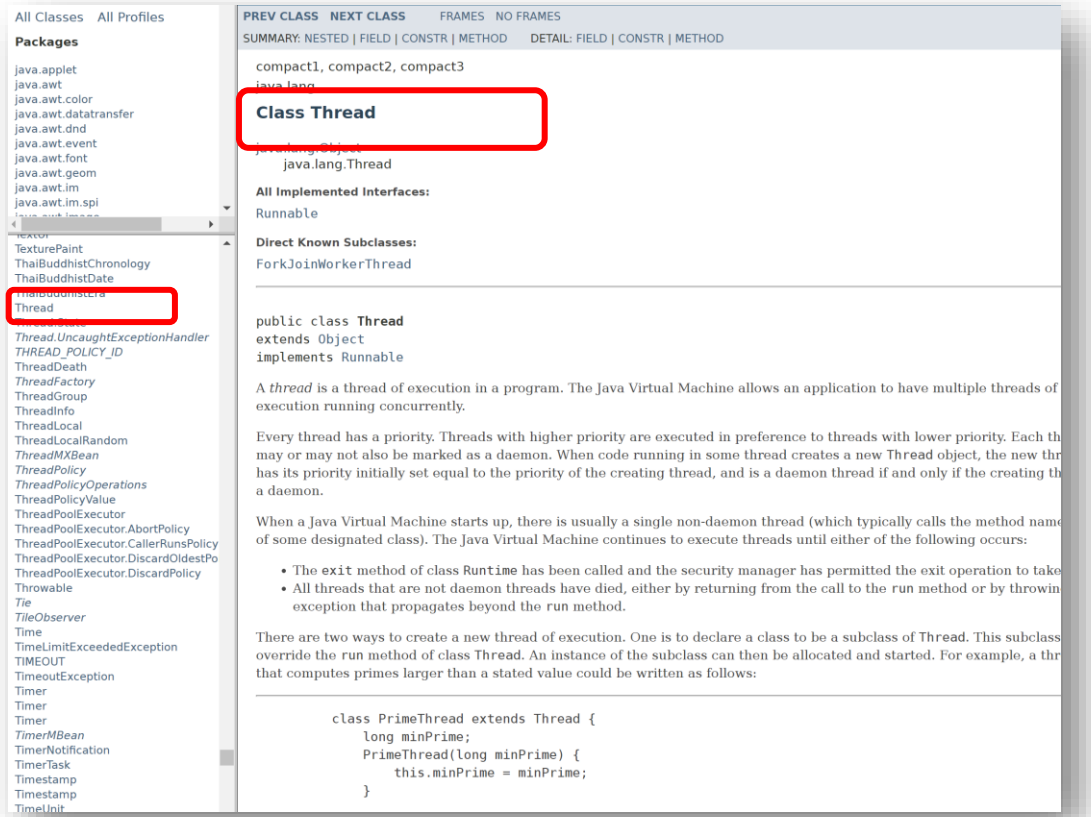
- Start thinking of which THREADS are needed to be executed synchronously or in parallel.
- Tip: start by implementing **thread_calibrate**, **thread_gotoX** and **thread_gotoZ**…
- Draw a first diagram illustrating the threads and the interactions between them.

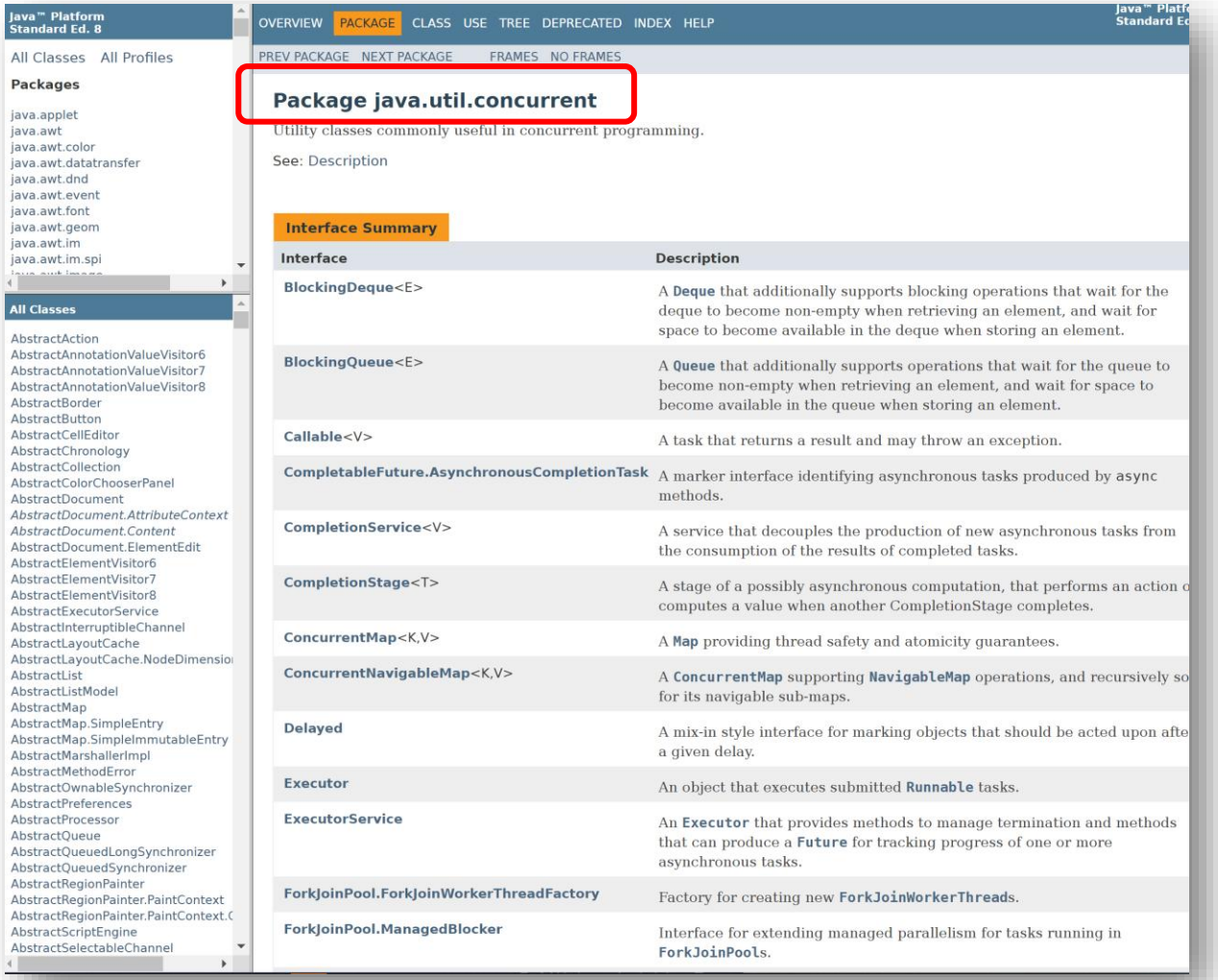
TO STUDY….

REVIEW

Check….

https://docs.oracle.com/javase/8/docs/api/index.html





https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html

THREADS – SOME ASPECTS TO CONSIDER

Two ways to start a thread (1)

```
@FunctionalInterface
public interface Runnable
```

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. The class must define a method of no arguments called run.

https://docs.oracle.com/javase/8/docs/api/index.html

In order to create **threads,** we need to provide a runnable object, as shown in the example.

Any Runnable class must implement the method **run().**

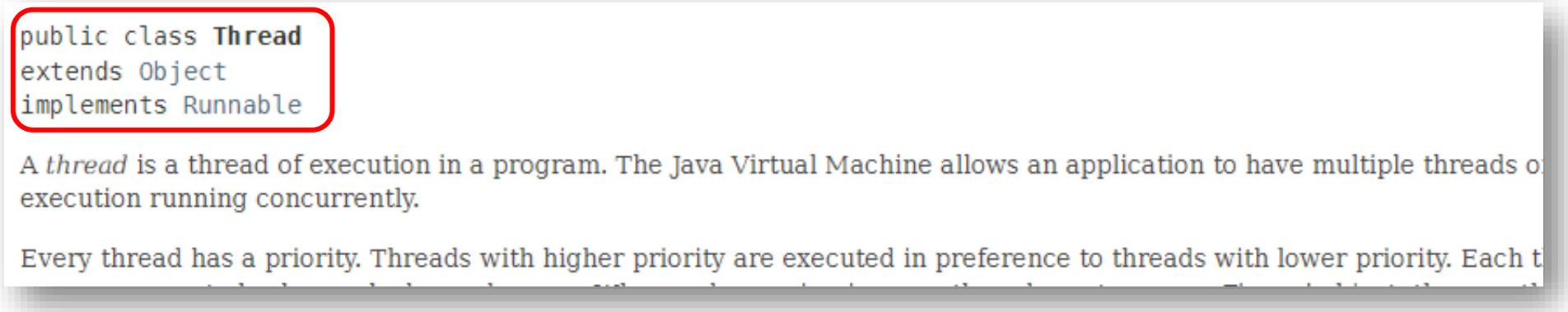
Notice that HelloRunnable has got this method.

```
public class HelloRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Hello from thread!");
    }
    public static void main(String[] args) {
        (new Thread(new HelloRunnable())).start();
    }
}
```

https://en.wikipedia.org/wiki/Java_concurrency

THREADS – SOME ASPECTS TO CONSIDER

Two ways to start a thread (2)



```
public class Thread
extends Object
implements Runnable
```

A *thread* is a thread of execution in a program. The Java Virtual Machine allows an application to have multiple threads o
execution running concurrently.

Every thread has a priority. Threads with higher priority are executed in preference to threads with lower priority. Each t

https://docs.oracle.com/javase/8/docs/api/index.html

In order to create threads, we need to implement a subclass of **Thread**.

```java
public class HelloThread extends Thread {
    @Override
    public void run() {
        System.out.println("Hello from thread!");
    }
    public static void main(String[] args) {
        (new HelloThread()).start();
    }
}
```

https://en.wikipedia.org/wiki/Java_concurrency

THREADS – ASPECTS TO CONSIDER

Waiting for threads termination: Thread.join

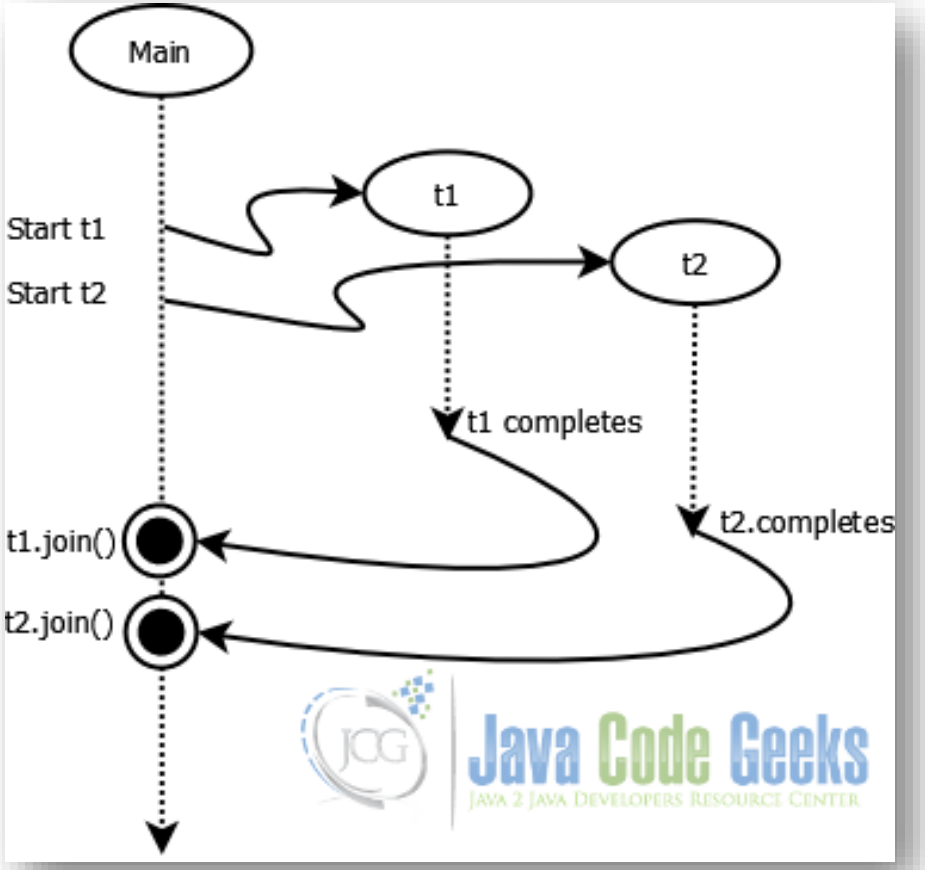
`join()`
Waits for this thread to die.

https://docs.oracle.com/javase/8/docs/api/index.html

- When **Thread.join** is called, it suspends the execution of the calling thread until the target thread is alive.

- Once the target thread is done with its job and terminates, the caller thread will automatically wake up.
  In the diagram, the caller thread starts two threads t1 and t2, each thread is assigned one specific task.

- The caller thread then calls on **t1.join** so that it is blocked from further execution till t1 finishes its job.
  Once t1 is done with its execution and terminates, it wakes up the caller thread.
  Next, caller thread calls **t2.join** to wait on t2 till it is done with its task.
  Once t2 finished, caller thread wakes up and continues in its execution.



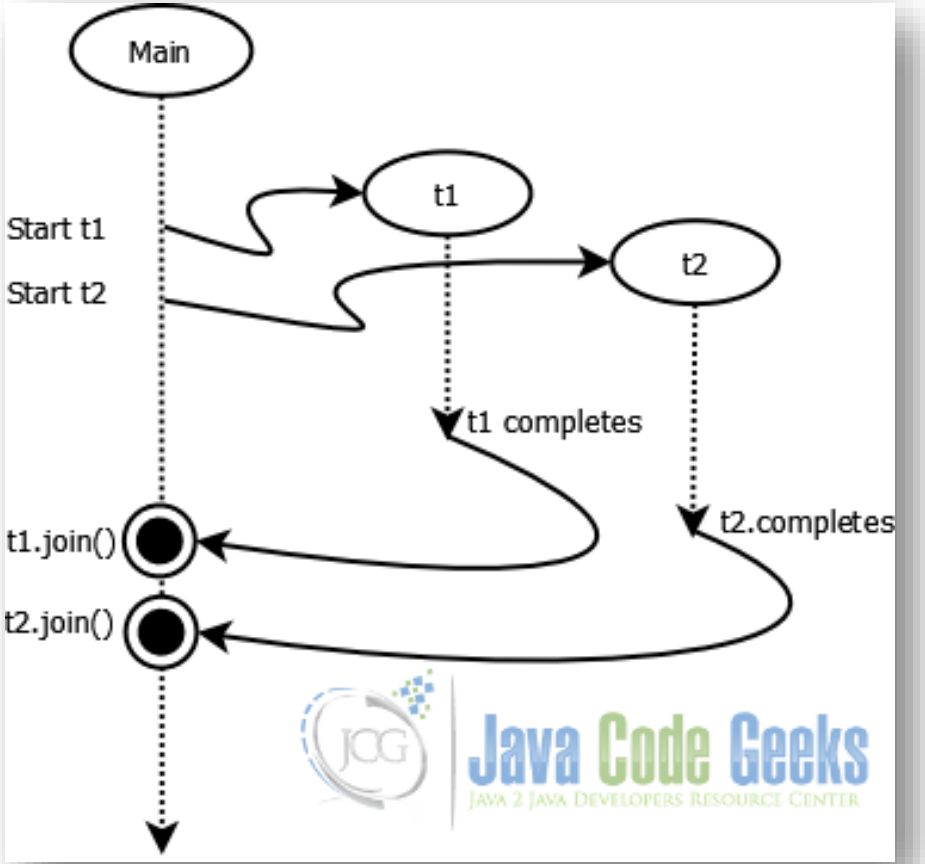
https://examples.javacodegeeks.com/core-java/threads/java-thread-join-example/

SHORT-LIVED THREADS

Threads which start execution, perform their job in a concurrent way, and complete….

Generally, they perform a single job then finish.

Additionally, we might **synchronize** them (as illustrated in the picture), but not strictly necessary….



https://examples.javacodegeeks.com/core-java/threads/java-thread-join-example/

SHORT-LIVED THREADS - EXAMPLES

Single-Thread (implicit MainThread)

```java
public class SortExample {
    public static int[] generateHugeArray(int size){
        Random rd = new Random(); //creating Random object
        int[] arr = new int[size];
        for (int i = 0; i < arr.length; i++){
            arr[i] = rd.nextInt(); // storing random integres in an array
        }
        return arr;
    }

    Run | Debug
    public static void main(String[] args){
        // Our arr contains 8 elements
        // int[] arr = {13, 8, 5, 78, 21, 7, 101, 156}

        int arr1[] = generateHugeArray(size:2000000);
        int arr2[] = generateHugeArray(size:2000000);

        long ti = System.currentTimeMillis();
        Arrays.sort(arr1);
        Arrays.sort(arr2);
        long tf = System.currentTimeMillis();

        System.out.printf(format:"Time : %d miliseconds", tf-ti);
    }
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS   Run: SortExample
 'C:\STR\Labwork2\labwork2\target\classes' 'str.SortExample'
Time : 577 miliseconds
PS C:\STR\Labwork2\labwork2>
```

Ln 31, Col 1 Spaces: 4 UTF-8 CRLF {} Java

Multi-Thread

```java
public class SortExample {
    public static int[] generateHugeArray(int size){
        Random rd = new Random(); //creating Random object
        int[] arr = new int[size];
        for (int i = 0; i < arr.length; i++){
            arr[i] = rd.nextInt(); // storing random integres in an array
        }
        return arr;
    }

    public static Thread sortParallel(int[] array){
        Thread worker = new Thread(){
            public void run(){
                Arrays.sort(array);
            }
        };
        worker.start();
        return worker;
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException{
        // Our arr contains 8 elements
        // int[] arr = {13, 8, 5, 78, 21, 7, 101, 156}

        int arr1[] = generateHugeArray(size:2000000);
        int arr2[] = generateHugeArray(size:2000000);

        long ti = System.currentTimeMillis();
        //Arrays.sort(arr1);
        //Arrays.sort(arr2);
        Thread worker1 = sortParallel(arr1);
        Thread worker2 = sortParallel(arr2);
        worker1.join();
        worker2.join();
        long tf = System.currentTimeMillis();

        System.out.printf(format:"Available processors : %d\n", Runtime.getRunt
        System.out.printf(format:"Time : %d miliseconds", tf-ti);
    }
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS   Run: SortExample
Available processors : 8
Time : 387 miliseconds
PS C:\STR\Labwork2\labwork2>
```

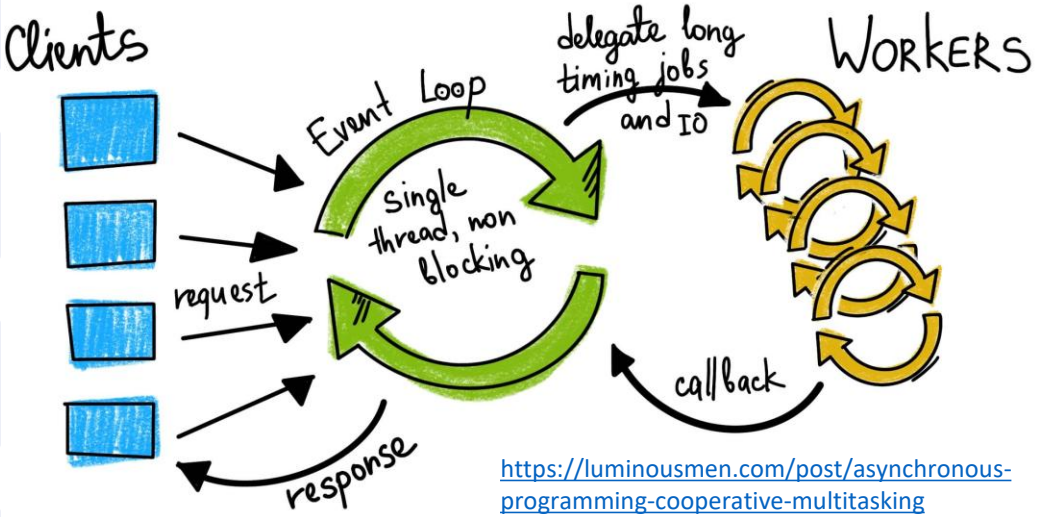
LONG-LIVED THREADS

Threads that after being created, stay handling the problems for a long time.

They are structured in a loop-like way, holding-on for a new Job/problem to work on,

…exhibiting multi-tasking cooperative, and reactive, behavior patterns

In many situations, long-lived threads are desirable, to reduce overhead for task creation, memory garbage collection…, less dynamic allocation



https://luminousmen.com/post/asynchronous-programming-cooperative-multitasking

LONG-LIVED THREADS - EXAMPLE

```java
public class ServerThread extends Thread{
    private boolean interrupted;

    public boolean isInterrupted(){
        return interrupted;
    }

    public void setInterrupted(boolean interrupted){
        this.interrupted = interrupted;
    }

    public void run(){
        this.setInterrupted(interrupted:false);
        while(!interrupted){
            //do something
            System.out.println(x:"doing some task here until interrupted");
            try {
                Thread.sleep(millis:1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
                Logger.getLogger(ServerThread.class.getName()).log(Level.SEVERE, msg:null, e);
            }
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException{
        ServerThread worker = new ServerThread();
        worker.start();
        Thread.sleep(millis:3000);
        worker.setInterrupted(interrupted:true);
        System.out.println(x:"Thread was interrupted");
    }
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Run: ServerThread

PS C:\STR\Labwork2\labwork2>  & 'C:\Users\faf\AppData\Local\Programs\Eclipse Adoptium\jdk-17.0.4.101-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\STR\Labwork2\labwork2\target\classes' 'str.ServerThread'
doing some task here until interrupted
doing some task here until interrupted
doing some task here until interrupted
Thread was interrupted
```

Usually in a **while(true) {…} loop** until interrupted

The **interrupted variable** can be set
internally (inside the loop), or
externally (in other part of the Java application)

```java
import java.util.concurrent.Semaphore;
import java.util.logging.Level;
import java.util.logging.Logger;

public class ServerThread extends Thread{

    private boolean interrupted = false;
    private Semaphore semaphoreSync = null;


    public ServerThread(){
        this.semaphoreSync = new Semaphore(permits:0);
    }

    public void signalWork() {
        semaphoreSync.release();
    }

    public boolean isInterrupted(){
        return this.interrupted;
    }

    // interrompe e acorda a thread caso esteja em acquire()
    public void setInterrupted(boolean interrupted){
        this.interrupted = interrupted;
        this.semaphoreSync.release();
    }

    public void run() {
        try{
            while(!this.isInterrupted()){
                this.semaphoreSync.acquire();
                if (this.isInterrupted()) break; // sai se já tiver sido interrompida
                System.out.println(x:"doing some task here until interrupted");
            }
        }catch(InterruptedException e){
                Logger.getLogger(ServerThread.class.getName()).log(Level.SEVERE, msg:null, e);
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException{
        ServerThread worker = new ServerThread();
        worker.start();

        // sinaliza algum trabalho
        worker.signalWork();
        worker.signalWork();
        worker.signalWork();

        // dá tempo à thread para consumir e imprimir
        Thread.sleep(millis:200);

        worker.setInterrupted(interrupted:true);
        worker.join();
        System.out.println(x:"Thread was interrupted");
    }

}
```

SYNCHRONIZATION WITH A SEMAPHORE

Using the previous example, we can use a Java semaphore to synchronize work.

After three **releases/signalWork**, the Thread blocks by the semaphore *acquire()*, until more releases.

In this example, a manually created *boolean* variable called **interrupted** is being used to control when the thread should stop.
However, Java already provides its own **built-in mechanism** for interrupting threads, using **Thread.interrupt()**, **isInterrupted()**, and the **InterruptedException**.

This approach is problematic because:

- The programmer's custom *boolean* is **not safely shared** between threads: one thread might change its value, but the other thread may never "see" that change.
- The thread can become **blocked** in operations like *acquire()* and never stop, even if the *boolean* value changes.
- The code **does not follow Java's best practices**, and it can easily lead to confusion or errors when you start using more advanced Java libraries and concurrency tools.

rewrite the code using Java's native interruption mechanism.

BLOCKING QUEUE EXAMPLE

```java
public class ServerThread2 extends Thread{
    private boolean interrupted;
    private ArrayBlockingQueue<Integer> queue = null;

    public ServerThread2(){
        this.queue = new ArrayBlockingQueue<Integer>(capacity:10);
    }

    public boolean isInterrupted(){
        return interrupted;
    }

    public void setInterrupted(boolean interrupted){
        this.interrupted = interrupted;
    }

    public ArrayBlockingQueue <Integer> getQueue(){
        return this.queue;
    }

    public void run(){
        this.setInterrupted(interrupted:false);
        while(!interrupted){
            try {
                int x = this.queue.take();
                System.out.printf(format:"\n got %d...", x);
            } catch (InterruptedException e) {
                Logger.getLogger(ServerThread2.class.getName()).log(Level.SEVERE, msg:null, e);
            }
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException{
        ServerThread2 worker = new ServerThread2();
        worker.start();
        ArrayBlockingQueue<Integer> q1 = worker.getQueue();
        q1.add(e:100);
        q1.add(e:200);
        q1.add(e:300);
        worker.setInterrupted(interrupted:true); // it does not work. why?
        System.out.println(x:"Thread was interrupted");
    }
}
```

The example illustrates a utilization of a blocking queue in a thread.

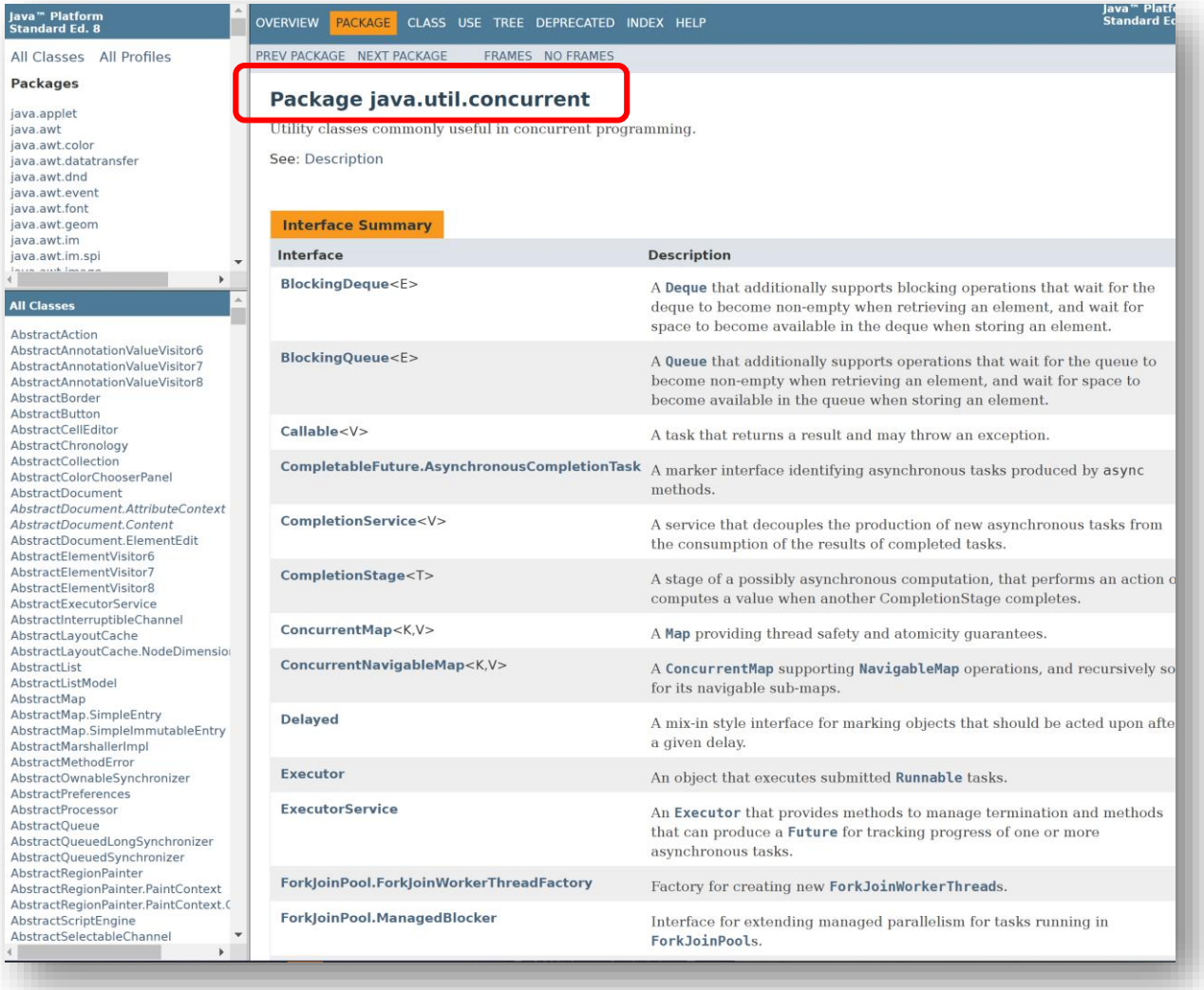
Inside the **run()** method, the thread keeps blocked in the **take()** method until a new message (an integer value) arrives.

The program does not work… why?? Look at the previous example….

BLOCKING QUEUE – OTHER EXAMPLES

From the JavaDocs:

- A ConcurrentLinkedQueue is an appropriate choice when many threads will share access to a common collection. This queue does not permit null elements.
- ArrayBlockingQueue is a classic "bounded buffer", in which a fixed-sized array holds elements inserted by producers and extracted by consumers. This class supports an optional fairness policy for ordering waiting producer and consumer threads
- LinkedBlockingQueue typically have higher throughput than array-based queues but less predictable performance in most concurrent applications.



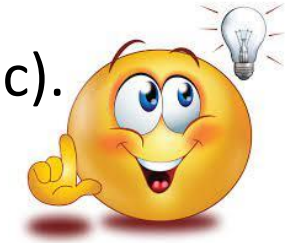
https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html

SUGGESTIONS FOR LAB WORK 2

Req.	Description
	Low-level requirements (DLL development):
RL1	User Keyboard interaction providing execution of actions and showing warehouse states.
RL2	Develop the **DLL** and expose low-level I/O functions through **JNI.**
	High-level requirements (Java Development)
RH1	*Calibration*. Develop robust (semi-automatic) storage calibration. During a movement, the actuator must stop when it reaches a position sensor. It should always initiate with the cage in position (X=1, Z=1, Y=2).
RH2	*Manual Storage*. Store a pallet in the storage. Ask for pallet metadata (product_type, humidity, producer_ID, destination, shipping date) and target cell (X,Z). Validate if cell is empty.
RH3	*Assisted storage*. When (X,Z) is not provided, invoke the "AI-Assisted Placement Module" recommender (balance occupancy, group by type, minimize travel distance). Based on log decisions.
RH4	*Deliver pallet by cell (X,Z)*. Retrieve the pallet from cell (X,Z) specified through keyboard and unload based on the active mode: **Mode 1:** unload at (X=3, Z=1) with Y=1. **Mode 2:** unload at (X, Z=1) with Y=1 and LED2 flashing during the process.
RH5	*Deliver by product or producer*. List and deliver matching pallets.
RH6	*Shipping and humidity alerts*. Flash **LED1** when a pallet's shipping date arrives or humidity exceeds a threshold (threshold may be defined by the student and must be documented).
RH7	*Mass removal (Switch 1)*. Remove all pallets with active alerts; LED1 flashes until process completes.
RH8	*Emergency STOP (Switch 1 + Switch 2)*. Emergency! Stop motion and freeze operations.
RH9	*Resume (Switch 1)*. During an emergency STOP, if switch 1 is pressed, then RESUME operation.
RH10	*Reset (Switch 2)*. During an emergency STOP, if switch 2 is pressed, then RESET the system (perform semi-automatic storage calibration).
RH11	*Emergency indicators*. Both LEDs flash rapidly (0.5 s) during emergency until resume/reset.
RH12	*Product list*. Display list of stored pallets (product types, producer IDs, humidity and shipping dates.
RH13	*Pallet lookup*. Display (X,Z), humidity, destination, and shipping info for selected product type/producer.

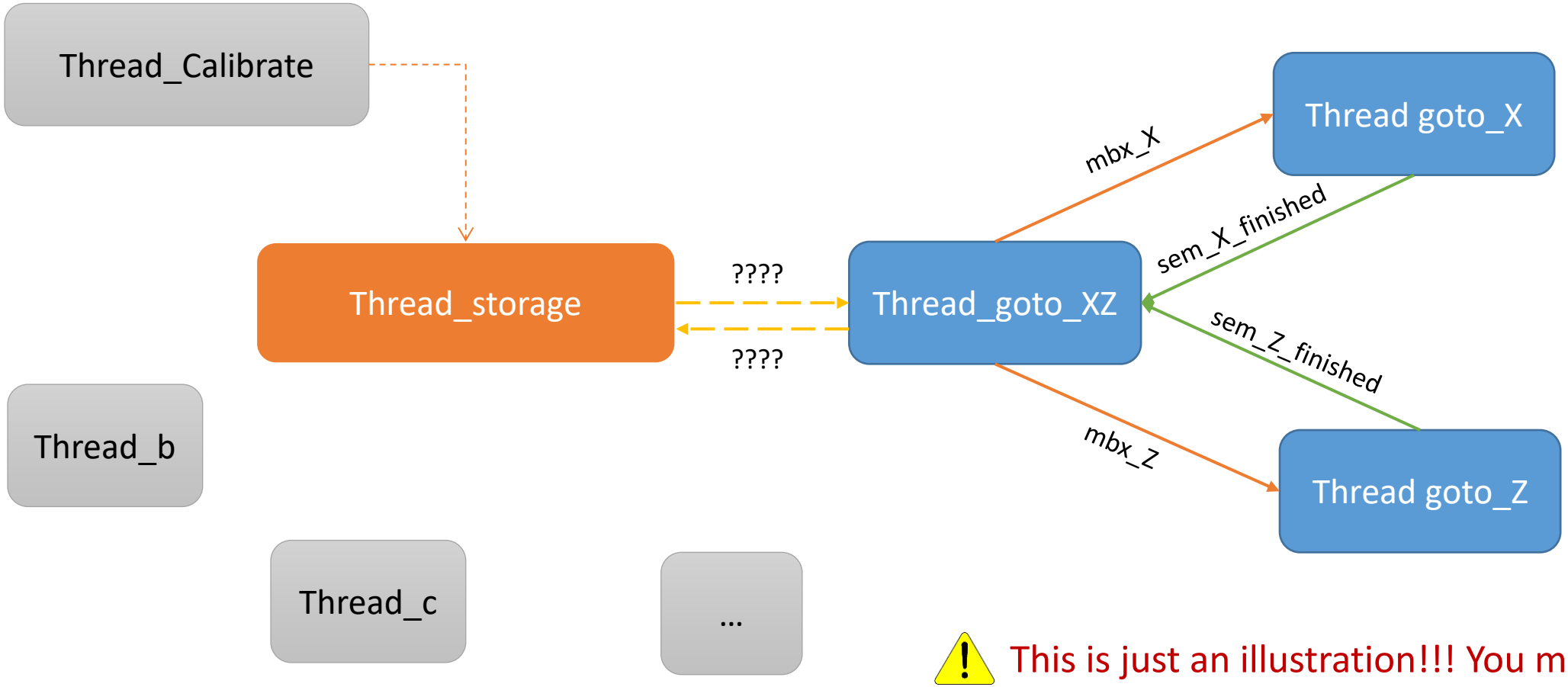
- Brainstorm your lab work to perceive which threads should be short lived or to persist.
- Could an information thread be a long-running one?

- Explore the java.util.concurrent and decide which concurrent mechanisms can be used to faithfully satisfy the requirements in the lab work.
- However, Keep it simple (but not simplistic).



LAB WORK 2 - STORAGE SYSTEM

- Let's start by designing a diagram of threads and inter-threads interactions:



⚠ This is just an illustration!!! You must adapt it according with your approach…

CALIBRATION THREAD

Aspects to consider….

Welcome | CalibrationThread.java

src > main > java > str > CalibrationThread.java > ...

```java
package str;

public class CalibrationThread extends Thread{
    private final Axis axis;

    public CalibrationThread(Axis axis) {
        this.axis = axis;
    }

    public void initializeCalibration(){
        if (axis.getPos() == -1) {
            axis.moveForward();
            while (axis.getPos() == -1 || axis.getPos() > 3) { }
            axis.stop();
            if (axis.getPos() > 1) {
                axis.gotoPos(pos:1);  //with exception of Y axis!!!
            }
        }
    }

    @Override
    public void run() {
        this.initializeCalibration();
    }
}
```

The class **CalibrationThread** might be defined as shown on the right.

Its constructor accepts an object that implements the interface Axis.

This means that **CalibrationThread** can accept and operate with any of the objects instantiated from class AxisX, AxisZ or AxisY (attention with AxisY, Y = 1 is not calibrated…).

In this example, regardless of which axis provided, after calling the start() method, the **CalibrationThread** should sequentially call getPos, moveForward, etc.. of the provided axis.

KEEP UP THE GOOD WORK!